

An Agent-based approach for the automatic generation of valid SysML v2 Models in industrial contexts

Eduardo Cibrián, **Jose Olivert**, Juan Llorens,
José María Álvarez Rodríguez

Computer Science and Engineering Department, UC3M

Madrid, 2025

Agenda

1. *Introduction*
2. *State of the Art*
3. *Methodology*
4. *Evaluation and Results*
5. *Conclusions*

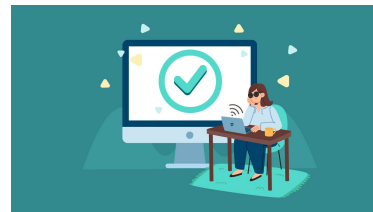
Introduction

MBSE offers a structured approach addressing the **complexity** in modern systems with **formal models**

Practical implementation of MBSE in industrial contexts remains a challenge.



How can AI and LLMs help this discipline?



Agent-based framework integrating **RAG techniques** and iterative syntactic and semantic validation for **automated SysML v2 model synthesis** from natural language specifications [1].

Introduction - *Contributions of this work*

- C1:** The design and application of an **intelligent agent architecture** that combines **LLMs**, **RAG** and **formal validation and synthesis**.
- C2:** A demonstration of how **retrieval-augmented prompts** based on real SysML v2 examples can substantially **improve the correctness** and relevance of the generated models.
- C3:** Empirical evaluation showcasing the **effectiveness** of the proposed approach in generating valid SysML v2 models from **natural language requirements**.

State of the Art - *Systems Engineering*

Multidisciplinary field focused on the **design**, **integration**, and **management** of complex systems throughout their life cycle [2][3]. It is used in domains involving **complex systems** (defence, telecommunications, medical fields).

Stimulates the
interoperability
between systems

Increase in the
demand.

State of the Art - SysML v2



Second iteration of
Systems Modeling
Language.

Representation of
structure and
behavior of the
systems.

Efficient design,
simulation and
validation.

Metamodeling:
Allows to define
new elements and
relationships in a
more simple way.

Parts can be specified
outside the context of a
specific part definition.

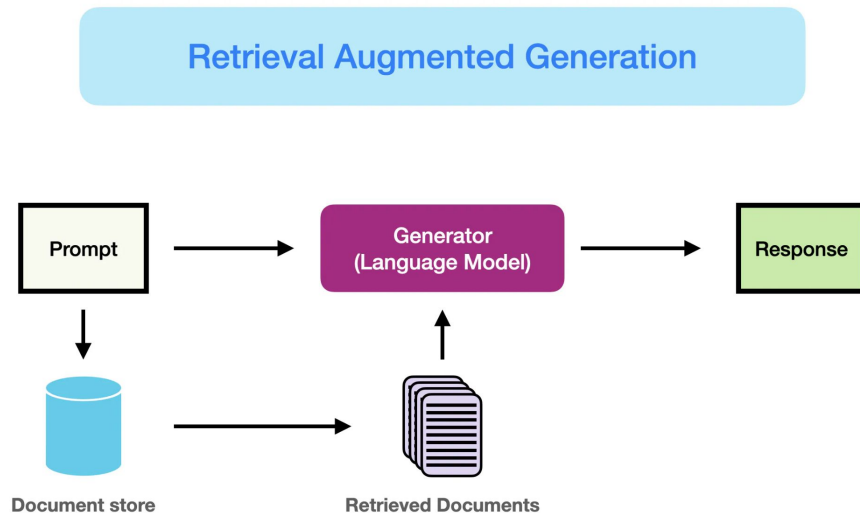
```
// Definitions
part def Vehicle {
    part eng : Engine;
}
part def Engine {
    part cyl : Cylinder[4..6];
}
part def Cylinder;

// Usages
part smallVehicle : Vehicle {
    part redefines eng {
        part redefines cyl[4];
    }
}
part bigVehicle : Vehicle {
    part redefines eng {
        part redefines cyl[6];
    }
}
```

The **defined by** relationship is a
kind of generalization.

Parts inherit properties from
their definitions and can
redefine them, to any level of
nesting.

State of the Art - *RAG*



Use of **vector databases**.

Mitigation of **hallucinations**
in LLMs [4].

State of the Art - *Positioning of the work*

Aspect	Existing Approaches	Proposed Methodology
Syntactic Validity	LLMs generate models with structural errors [5][6].	ANTLR-based validation ensures 100% syntactic compliance [7].
Semantic Grounding	Manual example retrieval or limited RAG implementations [8].	Structured model retrieval via domain-specific RAG [9].
Validation Integration	POst-hoc validation in separate tools [10].	Embedded validation loop with auto-correction [11].
Formal Language Support	Generic code generation focus [12].	SysML v2 specialization with grammar-aware synthesis.
Complexity Handling	Limited to component level modeling [13].	Supports system-of-system decomposition.

Methodology - SysML v2 Agent

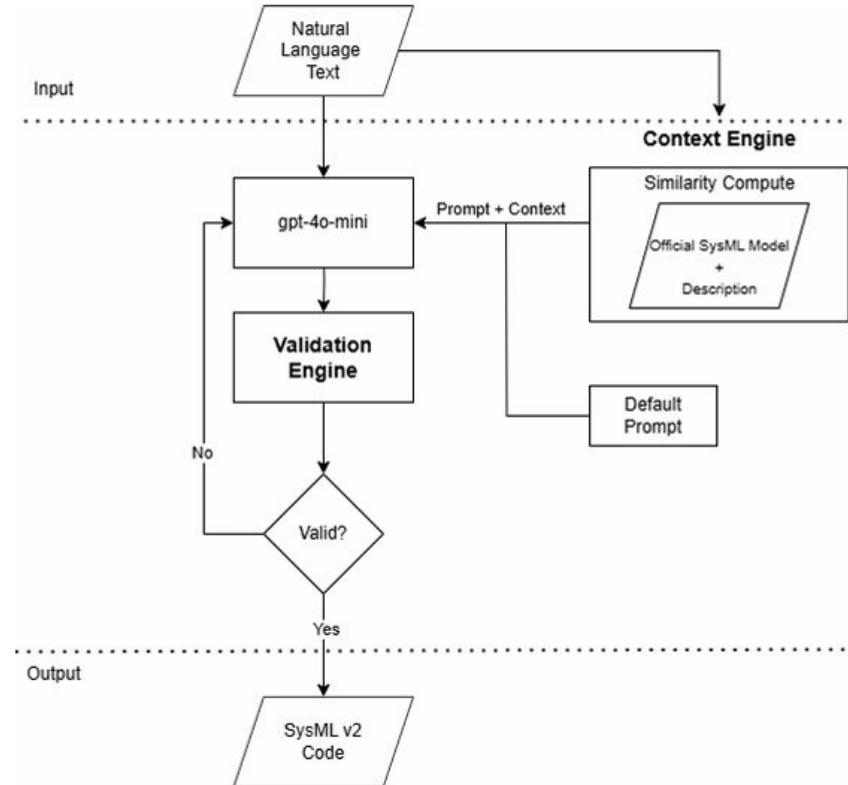
The functionality of the Agent can be defined as [14]:

$$\text{Agent} = (P, G, A, R, V)$$

- **P** represents the **perception**, interpretation of the natural language inputs.
- **G** denotes the **generative process of candidate models**.
- **A** comprehends the **set of actions**.
- **R** includes the **reasoning** mechanism.
- **V** refers to the **validation**.

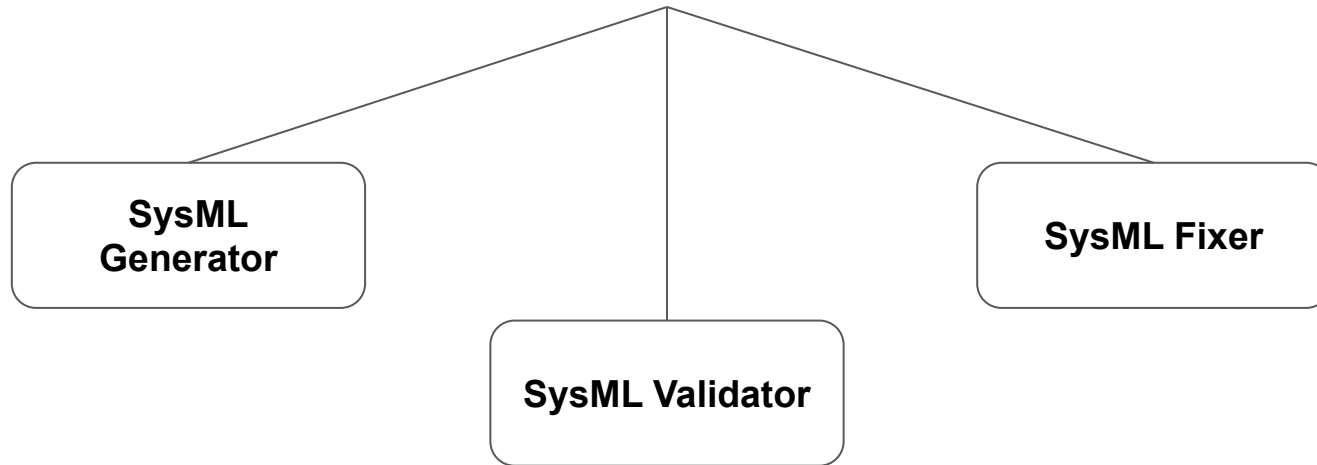
This design aligns with the principles of Belief-Desire-Intention (BDI) models.

Methodology - SysML v2 Agent



Methodology - SysML v2 Agent Toolset

A denotes the **set of actions**, called **tools**:



Methodology - SysML v2 Agent

Prompting Strategy

SysML Generator

System Prompt: *You are a helpful assistant who is in charge of creating SysML v2 models given an input from a user.*

User Prompt: *Given that user input, give me a valid SysML v2 model in that represents what the user wants. Return ONLY the SysML v2 code. In order to help you, I have extracted the two nearest models that we have in our local database to give you a little bit more of context. Remember, return me JUST the SysML v2 code generated.*

SysML Fixer

System Prompt: *You are an expert assistant in correcting SysML v2 models given the errors from the Validator.*

User Prompt: *Given the following SysML v2 code, correct that code taking into account the errors found. Return ONLY the corrected SysML v2 code.*

Methodology - SysML v2 Agent

Agent workflow and logic

Input: User input in natural language

Output: Valid SysML v2 model

begin

 Extract relevant model descriptions from the user input using the

Context Engine;

 Build a detailed prompt by combining the user input with the
 extracted contextual descriptions;

 Generate an initial SysML v2 model using the **SysML**

Generator with the constructed prompt;

 Validate the generated SysML v2 model using the **Validator**
 Engine;

while *Validation result is invalid* **do**

 Extract syntactic and semantic errors from the validator
 output;

 Construct a corrective prompt including the current model
 and the extracted errors;

 Regenerate an improved SysML v2 model using the **SysML**

Fixer with the corrective prompt;

 Re-validate the new SysML v2 model using the **Validator**
 Engine;

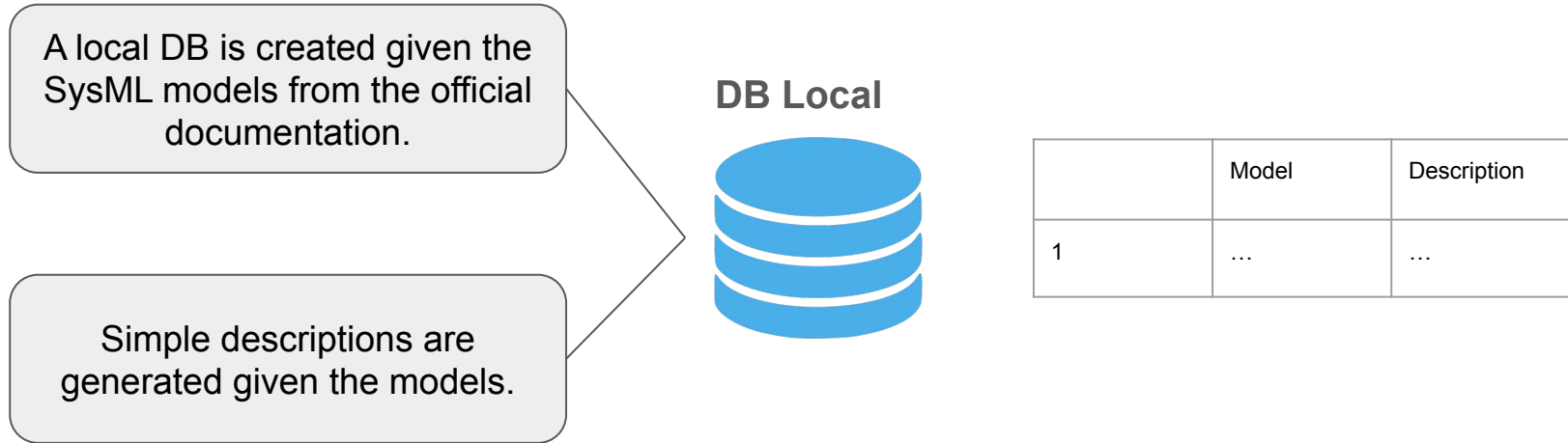
end

return Validated and correct SysML v2 model;

end

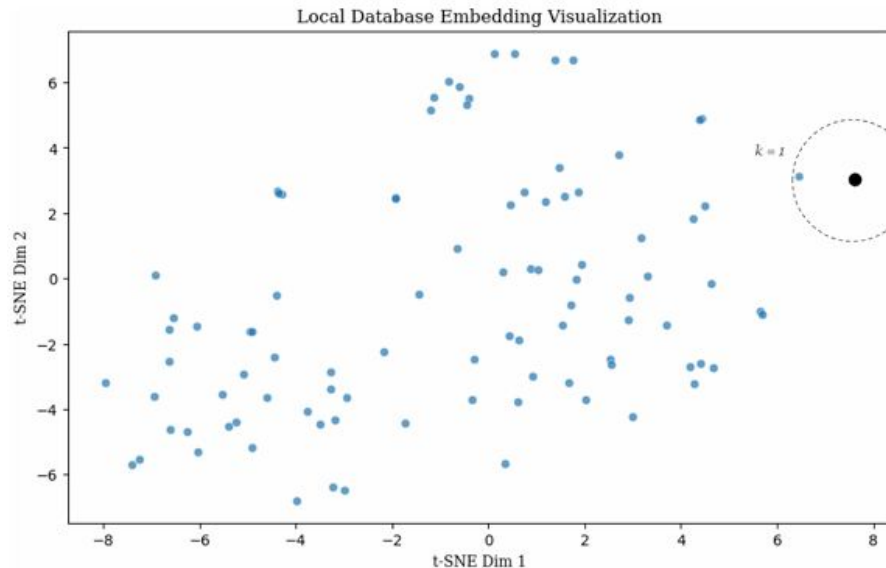
Algorithm 1: Pseudocode of the Agent Workflow.

Methodology - *Context Engine*



Methodology - *Context Engine*

Based on [Embedding and Cosine Similarity](#).



Embeddings model:
all-MiniLM-L6-v2

Description embeddings are
precomputed

Methodology - *Validation Engine*

ANTLR grammar is used.

Phase 1: Syntactic Validation. The model is divided in tokens and the parser verifies that they adhere to the language rules.

Phase 2: Semantic Validation. Verification of the logical coherence and conformity with context-dependent restrictions.

ANTLR provides lexer, visitor, parser and listener

Methodology - *Validation Engine*

Parse KerMLv2/SysMLv2 String

KML String:

```
1 package AvionicSystem {
2   private import SI::*;
3
4   block AvionicMotor {
5     attribute ratedPower :> ISQ::power;
6     attribute nominalVoltage :> ISQ::electricPotential;
7     attribute maximumSpeed :> ISQ::angularVelocity;
8
9     attribute def MotorInput :> Input {
10       attribute electricalPower :> ISQ::power;
11     }
12
13     attribute def MotorOutput :> Output {
14       attribute mechanicalTorque :> ISQ::torque;
15     }
16
17     attribute definition MotorState :> StateSpace {
18       attribute rotationalSpeed :> ISQ::angularVelocity;
19     }
20   }
21 }
```

Select Parser:

SysMLv2

Parse

Syntax error at line 4, column 10: extraneous input 'AvionicMotor' expecting {'default', '{', '}', '=', ':='}
Syntax error at line 17, column 29: mismatched input 'MotorState' expecting {'default', '{', '}', '=', ':='}

Evaluation and Results

For the purpose of evaluating the system, an **experimental setup** has been design to contemplate the different aspects of it.

Dataset of
Prompts

Selection of
different systems
under comparison

Configuration

Evaluation metrics

Evaluation and Results - *Dataset of Prompts*

The prompts were systematically designed to ensure comprehensive coverage of the core modeling construct and use cases illustrated within the official SysML v2 specification and associated examples [15].

ID	Description
U_1	Create a part called AvionicMotor. It uses electrical power to generate mechanical torque. It has three properties: rated power in kilowatts, nominal voltage in volts, and maximum speed in RPM.
U_2	Create a model for a drone with a battery, a motor, and an altitude sensor.
U_3	Create a central controller with a temperature sensor that is able to log data.
U_4	Design a robot arm that can rotate 360 degrees and has a gripper to pick up small objects.
U_5	Model a drone system with GPS navigation, a camera module, and an obstacle avoidance function.
U_6	Specify a water pump system that activates below 30% and deactivates at 90% tank level.
U_7	Describe a car that includes an electric motor, battery pack, and regenerative braking.
U_8	Create a coffee machine system with a user interface, water heater, and milk frother.
U_9	Define a process where a user logs into a mobile app and retrieves their profile data.
U_{10}	Model a smart home system that is able to turn on the lights.
U_{11}	Design a patient monitoring device that tracks vital signs and sends alerts.
U_{12}	Define a library management system that is able to borrow and return books.
U_{13}	Model a weather station that collects temperature, humidity, and wind speed data and is able to transmit it.
U_{14}	Create a requirement that an elevator stops at requested floors within 3 seconds.
U_{15}	Define an interface between a smartphone and a smart TV for screen mirroring functionality.
U_{16}	Model a vending machine that is able to dispose items.
U_{17}	Define the basic structure of a security system with a door sensor and an alarm.
U_{18}	Define a drone that has the ability to pick up objects from a structure.
U_{19}	Define a heating system that adjusts the temperature when there is a trigger.
U_{20}	Create a power unit that supplies electricity to servers and prevents overload.

Evaluation and Results - *Selection of different systems under comparison & Configuration*

SysML Agent: LLM
(GPT-4o-mini) with RAG and the
iterative validation loop

Baseline LLM 1: LLM
(GPT-4o-mini) without RAG and
the iterative validation loop

Baseline LLM 2: LLM (Gemini
2.5 Pro) without RAG and the
iterative validation loop

Agent No Rag: SysML Agent
without enabling the RAG
component

LLM Raw + RAG: LLM
(GPT-4o-mini) with the RAG
component

All LLM interactions used a consistent **low temperature setting** ($T=0.2$) to minimize the **output randomness**.

Evaluation and Results - *Evaluation Metrics*

Metric 1: Solved prompts (%). Quantifies the systems ability to produce a model output for a given prompt.

Metric 2: Syntactic validity rate (%). Measures the proportion of generated models that pass formal validation.

Metric 3: Manual quality score. Multilevel scale for semantic fidelity and completeness.

Evaluation and Results - *Evaluation Metrics*

Metric 3: Manual quality score

Score	Description of model quality
1	Low Fidelity / Incorrect: The generated model exhibits significant semantic misalignment with the prompt.
2	Partial Fidelity / Incomplete or Noisy: The generated model addresses the primary requirements of the prompt but suffers from notable omissions.
3	High Fidelity / Accurate and Complete: The generated model demonstrates high fidelity to the prompt.

Evaluation and Results - *Results*

System	Solved (%) (Prompts)	Syntactic Validity Rate (%)	Manual Quality (1-3)
SysML Agent	100	100	2.5
BL1	100	0	1
BL2	100	0	1
Agent No RAG	20	20	1
LLM Raw + RAG	100	50	1.9

Evaluation and Results - *Results*

```
package 'ElevatorSystem' {  
  requirement 'ElevatorStopTiming' {  
    attribute description : String := 'The elevator shall stop  
      at requested floors within 3 seconds of the request.';  
    attribute priority : String := 'High';  
    attribute status : String := 'Draft';  
  
    constraint 'StopTimingConstraint' {  
      attribute maxTime : ScalarValues::Real := 3.0;  
      attribute units : String := 'seconds';  
    }  
  }  
}
```

Create a requirement that the elevator shall stop at requested floors within 3 seconds of the request.

```
package 'UserLoginProcess' {  
  
  part def 'MobileApp';  
  part def 'User';  
  
  use case def 'Login' {  
    subject 'MobileApp';  
    actor 'User';  
  
    objective {  
    }  
  
    include use case 'AuthenticateUser' {  
      subject = 'MobileApp';  
      actor = 'User';  
    }  
  
    include use case 'RetrieveProfileData' {  
      subject = 'MobileApp';  
      actor = 'User';  
    }  
  }  
  
  use case def 'AuthenticateUser';  
  
  part 'User' : 'User';  
  
  use case 'RetrieveProfileData' {  
    actor :>> 'User';  
  }  
  
  use case 'LoginProcess' : 'Login';  
  
  part 'MobileApp' : 'MobileApp' {  
    include 'AuthenticateUser';  
    include 'RetrieveProfileData';  
    perform 'LoginProcess';  
  }  
}
```

Define a process where a user logs into a mobile app and retrieves their profile data.

Conclusions

C1: The combination of retrieval-augmented generation and validation creates a complementary system that significantly improves the syntactic and semantic correctness of SysML v2 models generated from natural language prompts.

C2: Future work will focus on scaling, optimizing latency, and integrating the system into industrial MBSE pipelines.

C3: The study reveals that general-purpose LLMs lack deep understanding of systems engineering languages like SysML v2, emphasizing the need for domain-specific augmentation through retrieval and structured validation.

References

- [1] Cibrián, E., Olivert Iserte, J. F., Casella, F., García Rodríguez, M., Alvarez-Rodríguez, J. M., & Llorens, J. An agent-based approach for the automatic generation of valid SysML v2 models from natural language specifications. Computers in Industry, 2025. DOI: 10.1016/j.compind.2025.104350.
- [2] INCOSE, SE Handbook V5, 2023.
- [3] NASA, Systems Engineering Handbook- NASA, 2018.
- [4] CelerData, Latest Developments in Retrieval-Augmented Generation, <https://celerddata.com/glossary/latest-developments-in-retrieval-augmented-generation>, 2025.
- [5] Zhu, X. et al. Model checking for code generation with LLMs. arXiv preprint arXiv:2212.09876, 2022.
- [6] Bonner, M. et al. LLM-based automation in systems engineering: A case study. Systems Engineering, 2024.
- [7] Parr, T. The definitive ANTLR 4 reference. The Pragmatic Bookshelf, 2013.
- [8] Patel, R. et al. RAG for code generation: Opportunities and challenges. arXiv preprint arXiv:2402.12345, 2024.
- [9] Lewis, P. et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in Neural Information Processing Systems, 2020.

References

- [10] Bajaj, M. et al. Systems modeling language (SysML) v2: Towards a next generation modeling language. INCOSE International Symposium, 32(1):1097–1112, 2022.
- [11] Luo, J. et al. Large language model agent: A survey on methodology, applications and challenges. arXiv preprint arXiv:2503.21460, 2025.
- [12] Ghaemi, M. et al. Transformers for code generation: A survey. IEEE Transactions on Neural Networks and Learning Systems, 2024.
- [13] Brown, K. et al. Model checking in MBSE: A survey. IEEE Transactions on Software Engineering, 2021.
- [14] S. Russell y P. Norvig, Artificial Intelligence: A Modern Approach, 4th US ed. 2021.
- [15] Object Management Group. OMG Systems Modeling Language (SysML) v2 Release. Available at: <https://github.com/Systems-Modeling/SysML-v2-Release>, 2023.

Thank you.

ecibrian@inf.uc3m.es, jolivert@inf.uc3m.es, juan.llorens@uc3m.es, joalvare@inf.uc3m.es